

Representing Adaptive Spacecraft Guidance Policies with Large Language Models in the Earth–Moon CRTBP

Daniel Amaya

August 26, 2026

Abstract

This paper investigates the feasibility and limitations of using a foundation language model as a language-conditioned spacecraft guidance policy within the Earth–Moon Circular Restricted Three-Body Problem (CRTBP). A nonlinear cislunar simulation framework is developed containing multiple trajectory classes, including Earth Orbits, lunar orbits, transfers, halo orbits, and distant retrograde orbits. An adaptive gain-scheduled PID-like controller generates disturbance-aware guidance trajectories, which are converted into a natural-language spacecraft-control dataset containing approximately 1.31 million prompt-response pairs. A Llama 3.1 8B Instruct model is fine-tuned using LoRA-based supervised imitation learning. The fine-tuned model is then evaluated as an LLM-driven sampled-data controller within the disturbed nonlinear CRTBP environment. The LLM produced commands that remained physically valid after output conditioning, but closed-loop simulations revealed increasing trajectory errors following disturbance injection. Analysis of this performance gap indicates that the natural-language representation captured observable spacecraft conditions and mission context, but did not explicitly encode several internal variables governing the adaptive PID-like controller’s behavior, including accumulated tracking-error history and adaptive feedback information. These results demonstrate that reproducing an adaptive guidance policy requires more than learning a state-action mapping; the representation must capture the complete information structure underlying the control decision process. Large language models therefore provide a flexible framework for mission-aware spacecraft guidance, but reliable autonomous operation will require control-oriented representations that incorporate latent controller states, temporal information, and trajectory-dependent dynamics.

1 Introduction

The increasing emphasis on sustained cislunar exploration has created a growing need for autonomous spacecraft guidance and control systems capable of operating with limited ground intervention. Future missions involving lunar transportation, communication relays, gateway logistics, and distributed deep-space infrastructure will require spacecraft to execute complex maneuvers across changing dynamical environments while maintaining safety, stability, and mission objectives.

Traditional spacecraft guidance architectures have demonstrated strong performance in these applications; however, many approaches rely on mission-specific trajectory design, extensive dynamical modeling, and carefully tuned control laws. This work investigates the feasibility and limitations of using a foundation language model as a language-conditioned spacecraft guidance policy within the Earth–Moon Circular Restricted Three-Body Problem (CRTBP). Rather than using a large language model only for mission planning or high-level reasoning, the proposed framework evaluates whether a fine-tuned LLM can directly generate continuous low-thrust acceleration

commands from structured descriptions of spacecraft state, reference trajectory information, tracking error, and mission context. The resulting model is evaluated as a closed-loop LLM-driven controller across multiple cislunar trajectory classes, including Earth-centered orbits, lunar orbits, Earth–Moon transfer trajectories, halo orbits, and distant retrograde orbits (DROs).

1.1 Autonomous Cislunar Navigation

Modern cislunar missions operate in dynamically complex environments that differ substantially from conventional low-Earth-orbit operations. Spacecraft may need to perform long-duration stationkeeping, trajectory corrections, and autonomous recovery maneuvers while communication latency and limited ground visibility restrict continuous human supervision. As mission architectures become more complex, onboard autonomous guidance systems must provide reliable trajectory tracking, disturbance rejection, propulsion constraint handling, and fault-tolerant operation.

Developing a single guidance architecture capable of operating across multiple mission phases remains challenging. Earth-centered trajectories, lunar trajectories, transfer paths, and libration-point orbits exhibit substantially different local dynamics and control requirements. Current approaches typically address these mission classes through specialized guidance solutions or carefully tuned control architectures designed around a specific trajectory family. Therefore, a key research challenge is determining whether a unified decision-making framework can provide useful guidance behavior across a diverse range of spacecraft operating conditions.

1.2 Complexity of Earth–Moon CRTBP Operations

The Earth–Moon Circular Restricted Three-Body Problem provides an important environment for studying autonomous cislunar guidance. The CRTBP describes the motion of a spacecraft under the gravitational influence of the Earth and Moon while assuming the spacecraft mass is negligible relative to the two primary bodies. Although simplified relative to a complete high-fidelity ephemeris model, the CRTBP captures many of the nonlinear behaviors that influence practical cislunar trajectories.

Trajectory families such as halo orbits, distant retrograde orbits, and nonlinear transfer trajectories are influenced by nonlinear gravitational coupling, sensitivity to initial conditions, and varying local dynamical characteristics. These characteristics create different guidance and stationkeeping challenges, particularly near libration-point regions, where small state deviations can result in significant long-term trajectory divergence. As a result, the CRTBP provides a useful benchmark for evaluating whether learned guidance policies can operate in environments where small errors can produce significant long-term trajectory deviations.

1.3 Large Language Models for Space-System Control

Recent advances in LLMs have demonstrated the ability of large language models to process complex information, follow instructions, and generate context-dependent responses across diverse applications. These models have been adapted for tasks including robotics, scientific computing, autonomous systems, and specialized engineering workflows. Unlike many traditional machine-learning policies that operate directly on numerical state vectors, LLMs provide a flexible interface capable of combining numerical information with semantic descriptions of objectives and operating conditions.

Recent work by Zucchelli et al. investigated fine-tuned language models as controllers for space-system applications, demonstrating that language models can learn control behaviors from PID-like-controller-generated demonstrations and produce structured control outputs for spacecraft scenar-

ios [3]. These results suggest that LLMs may provide a new mechanism for representing spacecraft decision-making problems through language-based state and action descriptions. However, previous demonstrations do not fully address the challenges associated with continuous control in nonlinear unstable dynamical environments. In particular, the ability of a language model to directly replace a feedback controller within a disturbed CRTBP simulation remains an open question.

1.4 Research Motivation

The primary motivation of this work is to evaluate whether spacecraft guidance can be represented as a language-conditioned imitation-learning problem. Classical controllers operate efficiently on numerical state vectors and provide explicit mathematical structures for stability analysis. Large language models, in contrast, provide a flexible representation capable of incorporating mission objectives, trajectory information, and additional contextual information within a unified input format.

This work examines the capabilities and limitations of deploying a fine-tuned language model as a direct LLM-driven closed-loop guidance policy. To perform this evaluation, a complete Earth–Moon CRTBP simulation environment was developed. An adaptive gain-scheduled PID-like feedback controller with integral compensation, thrust saturation, and anti-windup protection was implemented as the guidance policy used to generate trajectories. These demonstrations were converted into a large-scale natural-language dataset and used to fine-tune a Llama 3.1 8B Instruct LLM using LoRA-based supervised imitation learning. The resulting model was then deployed directly inside the spacecraft closed-loop guidance cycle and evaluated under nonlinear CRTBP dynamics with continuous disturbances and injected state perturbations. This approach enables examination of both the promise and the limitations of language-conditioned spacecraft control.

1.5 Contributions

The primary contributions of this work are:

1. Development of a nonlinear Earth–Moon CRTBP simulation framework supporting multiple cislunar trajectory classes, including Earth-orbit, lunar-orbit, transfer, halo, and DRO missions.
2. Design and validation of an adaptive PID-like guidance controller used to generate disturbance-aware trajectories across multiple cislunar trajectory classes.
3. Construction of a large-scale language-based spacecraft-control dataset containing over 1.3 million prompt-response pairs representing spacecraft states, mission context, and continuous three-axis acceleration commands.
4. Fine-tuning and closed-loop evaluation of a foundation language model as a direct LLM-driven closed-loop guidance policy, including quantitative analysis of the gap between language-model imitation performance and spacecraft-level trajectory-control performance.

The central research question investigated throughout this paper is whether a foundation language model can learn meaningful spacecraft guidance behavior from PID-like control trajectories and operate within a nonlinear cislunar control loop. The objective is not only to measure command-generation capability, but also to evaluate whether the learned language representation captures the temporal and dynamic characteristics required for reliable long-duration spacecraft guidance. The resulting analysis identifies both the capabilities of foundation-model-based guidance and the challenges that remain before such models can support fully autonomous spacecraft control.

2 Earth–Moon CRTBP Simulation Environment

The spacecraft guidance framework developed in this work is built around a nonlinear Earth–Moon Circular Restricted Three-Body Problem (CRTBP) simulation environment. The environment provides the dynamical model used for reference-trajectory generation, controller implementations, dataset construction, and closed-loop evaluation of the language-model guidance policy. The implementation combines nonlinear spacecraft propagation, state-transition-matrix computation, periodic-orbit correction, disturbance modeling, and multiple cislunar trajectory families within a common canonical-unit framework.

The CRTBP equations of motion implemented in `crtbp_eom()` within `CRTBP.py` follow the standard rotating-frame formulation described by Prussing and Conway [1].

2.1 Nonlinear Earth–Moon CRTBP Dynamics

The Earth–Moon CRTBP models spacecraft motion under the gravitational influence of the Earth and Moon while assuming that the spacecraft mass is negligible relative to the two primary bodies. The two primary bodies are assumed to move in circular orbits about their mutual barycenter, and the spacecraft equations are expressed in the rotating synodic reference frame. The nondimensional state vector is defined as

$$\mathbf{x} = [x \ y \ z \ \dot{x} \ \dot{y} \ \dot{z}]^T.$$

The nonlinear equations of motion are

$$\ddot{x} - 2\dot{y} = U_x, \quad (1)$$

$$\ddot{y} + 2\dot{x} = U_y, \quad (2)$$

$$\ddot{z} = U_z, \quad (3)$$

where the effective potential is

$$U(x, y, z) = \frac{1}{2}(x^2 + y^2) + \frac{1 - \mu}{r_1} + \frac{\mu}{r_2}. \quad (4)$$

The Earth and Moon distances are

$$r_1 = \sqrt{(x + \mu)^2 + y^2 + z^2}, \quad (5)$$

$$r_2 = \sqrt{(x - 1 + \mu)^2 + y^2 + z^2}, \quad (6)$$

where the Earth–Moon mass parameter is

$$\mu = 0.012150585609262.$$

The resulting nonlinear acceleration terms are implemented directly in `crtbp_eom()` and propagated numerically using adaptive Runge–Kutta integration through the SciPy `solve_ivp()` solver.

During controlled simulations, the nonlinear dynamics are augmented by spacecraft acceleration commands and continuous disturbance terms:

$$\dot{\mathbf{x}} = f_{\text{CRTBP}}(\mathbf{x}) + B\mathbf{u} + \mathbf{d}(t, \mathbf{x}), \quad (7)$$

where $B\mathbf{u}$ represents the commanded spacecraft acceleration and $\mathbf{d}(t, \mathbf{x})$ represents unmodeled disturbance acceleration.

2.2 Canonical Units and Numerical Propagation

To maintain numerical consistency across mission classes with substantially different spatial scales, all CRTBP simulations are performed using canonical units. The distance unit is selected as the average Earth–Moon separation:

$$\text{DU} = 384,400 \text{ km.}$$

The characteristic time unit is

$$\text{TU} = 375,700 \text{ s,}$$

which corresponds to the rotational timescale of the Earth–Moon system. The resulting velocity and acceleration conversions are

$$\text{VU} = \frac{\text{DU}}{\text{TU}}, \quad (8)$$

$$\text{AU} = \frac{\text{DU}}{\text{TU}^2}. \quad (9)$$

For the implemented system,

$$\text{VU} \approx 1023.15 \text{ m/s,}$$

and

$$\text{AU} \approx 2.723 \times 10^{-3} \text{ m/s}^2.$$

The nonlinear trajectory propagations use adaptive Runge–Kutta integration. Standard reference trajectories are generated using the `generate_reference_trajectory()` function in `Trajectories.py`, which uses RK45 propagation with adaptive error control and a maximum integration step of 0.005 canonical time units. Continuous reference trajectory access is provided through cubic-spline interpolation, allowing both the PID-like controller and LLM controller to query the desired state at arbitrary times.

The Jacobi constant,

$$C = 2U - v^2, \quad (10)$$

is an invariant of the unforced CRTBP and provides a useful theoretical measure of the system’s energy structure. Although this quantity is not used directly in the current controller or LLM evaluation, it provides physical context for analyzing the permissible regions of motion within the Earth–Moon system.

2.3 State Transition Matrix and Trajectory Correction

The simulation framework includes the capability to propagate the CRTBP variational equations alongside the spacecraft dynamics. The function `crtbp_var_eom()` in `CRTBP.py` integrates both the spacecraft state and the state transition matrix (STM),

$$\Phi(t, t_0),$$

which satisfies

$$\dot{\Phi} = A(\mathbf{x})\Phi, \quad (11)$$

where $A(\mathbf{x})$ is the local Jacobian of the CRTBP dynamics. The second partial derivatives required to form the local Jacobian matrix are computed using the `second_partials()` function.

The STM is used primarily for numerical trajectory correction through the multiple-shooting method implemented in `multiple_shooting_corrector()` within `Trajectories.py`. For a set of trajectory nodes,

$$X_0, X_1, \dots, X_N,$$

the multiple-shooting algorithm minimizes continuity defects between separately propagated trajectory segments. The correction process iteratively updates the node states using the integrated STM until the continuity residual satisfies the prescribed convergence tolerance. This correction procedure is used to improve the periodicity of the generated halo and DRO trajectories.

The simulation framework also contains optional invariant-manifold computation tools through `compute_halo_manifolds()` in `CRTBP.py`; however, these structures are not used as direct inputs to the current language-model policy.

2.4 Reference Trajectory Families

The simulation environment contains six trajectory categories used throughout dataset generation and controller evaluation:

- Earth Orbit,
- Moon Orbit,
- Earth–Moon Transfer,
- L_1 Halo Orbit,
- L_2 Halo Orbit,
- Distant Retrograde Orbit.

Earth-orbit, Moon-orbit, and transfer trajectories are generated through the `generate_reference_trajectory()` function. These trajectories use predefined initial conditions with optional perturbations and are propagated through the nonlinear CRTBP equations. Halo trajectories are generated using `generate_halo_orbit()` with numerical Lagrange-point locations obtained from `compute_lagrange_points()`. An analytical initial seed is first generated around either the L_1 or L_2 region and then corrected using the multiple-shooting differential-correction routine. Distant retrograde orbits are generated using `generate_dro_orbit()`. Similar to the halo-orbit process, an initial analytical seed is propagated and subsequently corrected numerically to improve periodic trajectory continuity. Together, these trajectory families provide a diverse set of dynamical conditions for training and evaluating the language-conditioned guidance architecture.

2.5 Continuous Disturbance Model

To evaluate disturbance-rejection behavior, continuous deterministic disturbance accelerations are included during both PID-like controller simulations and LLM closed-loop evaluations. The disturbance implementation is contained in the `unmodeled_perturbations()` function within `Trajectories.py`.

The disturbance model contains two components. First, a small radial acceleration term is included to approximate solar-radiation-pressure-like forcing:

$$a_{\text{SRP}} \approx 10^{-6} \text{ m/s}^2.$$

Second, deterministic multi-frequency harmonic acceleration components are applied independently along the three synodic coordinate axes. The harmonic terms are smoothly varying rather than random discontinuous impulses, allowing compatibility with adaptive numerical integration. Although simplified relative to a complete spacecraft disturbance model, this approach provides a controlled and repeatable method for testing whether the learned guidance policy can respond to persistent off-nominal conditions.

3 Adaptive PID-like Guidance and Control Architecture

The PID-like guidance controller provides the reference policy used to generate the supervised trajectories for foundation-model training. The controller is designed for nonlinear Earth–Moon CRTBP trajectory tracking and combines local dynamics linearization, adaptive gain scheduling, integral compensation, thrust saturation, and anti-windup protection. The adaptive gain-scheduled PID-like controller is not the primary learning contribution of this work; rather, it provides a physically validated source of guidance behavior from which the language-conditioned policy is trained. The controller implementation is contained in `Controller.py`, with the nonlinear dynamics and trajectory propagation routines provided through `CRTBP.py` and `Trajectories.py`. The control architecture follows established adaptive and robust-control principles described by Lavretsky and Wise [2], with modifications for disturbed Earth–Moon CRTBP trajectory tracking.

3.1 Local Linearized CRTBP Control Model

Although spacecraft propagation is performed using the full nonlinear CRTBP equations, the controller uses a continuously updated local linear approximation around the current reference trajectory state:

$$\mathbf{x}_{\text{ref}} = [x \ y \ z \ \dot{x} \ \dot{y} \ \dot{z}]^T.$$

The local tracking dynamics are approximated as

$$\delta \dot{\mathbf{x}} = A_p \delta \mathbf{x} + B_p \mathbf{u},$$

where A_p is the local CRTBP Jacobian and B_p represents the acceleration input matrix. The plant Jacobian is

$$A_p = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ U_{xx} & U_{xy} & U_{xz} & 0 & 2 & 0 \\ U_{xy} & U_{yy} & U_{yz} & -2 & 0 & 0 \\ U_{xz} & U_{yz} & U_{zz} & 0 & 0 & 0 \end{bmatrix},$$

with

$$B_p = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The Jacobian is updated continuously using the reference trajectory state, allowing the controller to adapt its feedback behavior as the spacecraft moves between different dynamical regions, including Earth-centered trajectories, lunar trajectories, transfers, and libration-point environments.

3.2 Adaptive Gain-Scheduled Feedback

The controller operates on the tracking error $\mathbf{e} = \mathbf{x} - \mathbf{x}_{\text{ref}}$, with position and velocity components:

$$\mathbf{e} = \begin{bmatrix} \mathbf{e}_p \\ \mathbf{e}_v \end{bmatrix}.$$

The commanded acceleration before saturation is calculated as

$$\mathbf{u}_{\text{raw}} = K_I \mathbf{x}_c - K_P \mathbf{e}_p - K_D \mathbf{e}_v,$$

where K_P , K_D , and K_I are proportional, derivative, and integral gains, and $\mathbf{x}_c$ is the accumulated integral error state.

To increase corrective authority in dynamically sensitive regions, the controller schedules the feedback gains according to the largest real eigenvalue of the local plant:

$$\lambda_{\max} = \max_i (\text{Re}(\lambda_i(A_p))) .$$

When $\lambda_{\max} > 0.5$, the proportional and derivative gains are increased using:

$$\Delta\lambda = \lambda_{\max} - 0.5, \quad s = 1 + 4\Delta\lambda + 3(\Delta\lambda)^2, \quad \text{where } 1 \leq s \leq 10.$$

The scheduled gains are $K_P = sK_{P,0}$ and $K_D = s^{0.6}K_{D,0}$, while the integral gain remains nominal ($K_I = K_{I,0}$). This approach provides increased feedback authority in locally unstable regions while maintaining lower control effort in comparatively stable regions.

3.3 Integral Compensation and Anti-Windup Protection

Integral compensation is included to reduce steady-state tracking error caused by persistent disturbance forcing. The controller maintains the integral state

$$\mathbf{x}_c = \begin{bmatrix} x_{c,x} \\ x_{c,y} \\ x_{c,z} \end{bmatrix},$$

with nominal propagation $\dot{\mathbf{x}}_c = -\mathbf{e}_p$.

Because large disturbances can cause excessive integral accumulation, anti-windup protection is incorporated through three mechanisms:

1. Reduction of integral accumulation during thrust saturation.
2. Freezing of the integrator when position error exceeds approximately 100 km.
3. Leaky clamping of the integral state when its magnitude exceeds the allowable bound.

These protections allow disturbance rejection while preventing integral growth from degrading recovery performance.

3.4 Smooth Thrust Saturation

The commanded acceleration is limited by the maximum spacecraft acceleration $u_{\max} = 0.01 \text{ m/s}^2$. Instead of component-wise clipping, the controller uses smooth vector-magnitude saturation:

$$\mathbf{u}_{\text{sat}} = \mathbf{u}_{\text{raw}} \frac{u_{\max} \tanh(\|\mathbf{u}_{\text{raw}}\|/u_{\max})}{\|\mathbf{u}_{\text{raw}}\|}.$$

This formulation preserves thrust direction while providing a continuous transition between unsaturated and saturated operation, improving compatibility with numerical propagation.

3.5 PID-like Trajectory Generation

The validated controller is used to generate the state-action trajectories used during dataset construction. The controlled nonlinear dynamics are represented as

$$\dot{\mathbf{x}} = f_{\text{CRTBP}}(\mathbf{x}) + B\mathbf{u}_{\text{sat}} + \mathbf{d}(t, \mathbf{x}),$$

where the disturbance term $\mathbf{d}(t, \mathbf{x})$ represents the continuous unmodeled acceleration model introduced in Section 2. For each demonstration, the spacecraft begins on the reference trajectory, undergoes continuous disturbance forcing, and may experience an injected state perturbation. The corresponding spacecraft states, tracking errors, and PID-like controller acceleration commands are stored for language-model training.

3.6 PID-like Controller Validation

The adaptive gain-scheduled PID-like controller was evaluated across five representative mission classes:

- Earth Orbit,
- Moon Orbit,
- Earth–Moon Transfer,
- L_1 / L_2 Halo Orbit,
- Distant Retrograde Orbit.

Each mission included nominal disturbed tracking followed by an injected position and velocity perturbation to evaluate both disturbance rejection and recovery behavior.

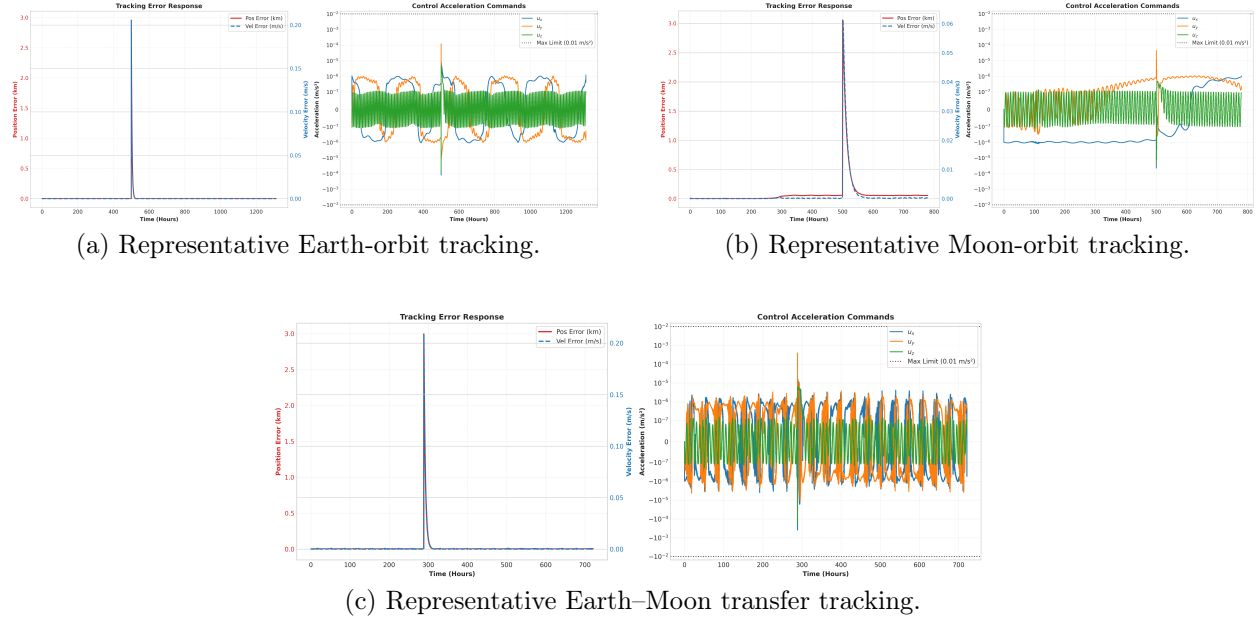


Figure 1: Trajectory tracking and disturbance-recovery performance across planetary and transfer mission classes using the PID-like controller.

The Earth-orbit, lunar-orbit, and transfer cases demonstrated effective disturbance rejection and maintained bounded tracking behavior. The transfer scenario exhibited larger transient variations due to changes in the local gravitational environment; however, the controller maintained bounded tracking behavior and produced smooth corrective accelerations.

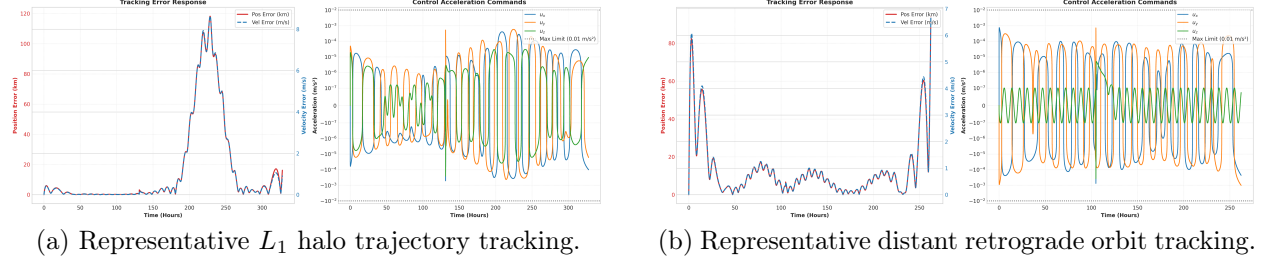


Figure 2: Trajectory tracking and disturbance-recovery performance across libration-point and periodic orbit classes using the PID-like controller.

The halo and DRO scenarios required greater control authority due to their dynamically sensitive operating regions. These cases resulted in higher cumulative correction effort than the planetary-orbit trajectories, demonstrating the increased challenge associated with maintaining weakly stable and libration-point-related trajectories.

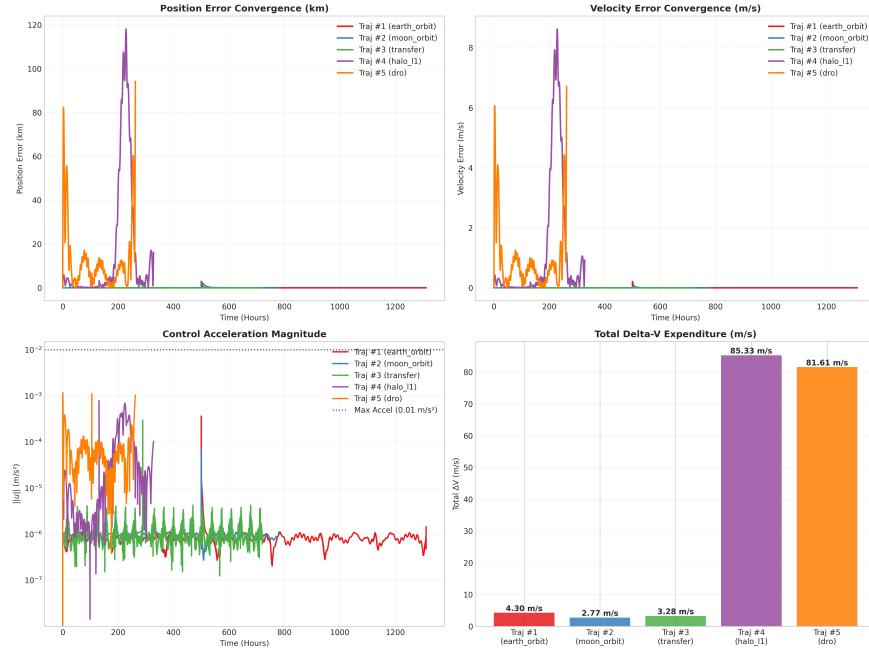


Figure 3: Aggregated PID-like controller performance comparison across Earth-orbit, lunar-orbit, transfer, halo, and distant retrograde orbit mission classes.

Mission Class	Total ΔV (m/s)
Earth Orbit	4.30
Moon Orbit	2.77
Earth–Moon Transfer	3.28
L_1 Halo Orbit	85.33
Distant Retrograde Orbit (DRO)	81.61

Table 1: Controller baseline control expenditure (ΔV) across evaluated cislunar mission classes.

Across all evaluated mission classes, the adaptive gain-scheduled PID-like controller maintained bounded acceleration commands and completed the prescribed simulations without safety violations or numerical instability. The consistent recovery performance demonstrates that the adaptive controller provides a reliable source of guidance across a diverse set of cislunar dynamical environments. These validated trajectories and corresponding control histories serve as the PID-like policy foundation for the natural-language dataset construction described in Section 4.

4 Natural-Language Spacecraft Guidance Dataset Construction

The proposed language-conditioned imitation-learning framework requires a dataset containing both spacecraft state information and corresponding PID-like guidance actions. Rather than representing guidance as a conventional numerical state-action learning problem, this work reformulates spacecraft control as a language-conditioned decision-making task. Spacecraft states, reference trajectory information, mission context, and tracking errors are converted into structured natural-language prompts, while the PID-like controller acceleration command is converted into a natural-language response. The dataset-generation pipeline is implemented in `Dataset_Generator.py` using the reference trajectory and controlled propagation tools developed in `Trajectories.py`. Each sample represents a physically propagated spacecraft state paired with the PID-like controller acceleration command required to maintain or recover the desired trajectory.

4.1 Reference Trajectory Generation

Dataset generation begins by producing reference trajectories across six cislunar mission categories:

- **Cislunar Orbits:** Earth Orbit, Moon Orbit, L_1 Halo Orbit, L_2 Halo Orbit, and Distant Retrograde Orbit (DRO).
- **Transfers:** Earth–Moon Transfer.

Earth-orbit, Moon-orbit, and transfer references are generated using the nonlinear CRTBP trajectory generator described in Section 2. Halo and DRO references are generated using corrected periodic trajectories produced by the multiple-shooting procedures in `Trajectories.py`. These trajectories provide continuous state histories for both the PID-like controller and the foundation-model training pipeline.

For halo and DRO missions, corrected node solutions are converted into dense continuous trajectories through nonlinear propagation and cubic-spline interpolation. A continuous reference function is then created, allowing the spacecraft state and desired state to be queried at arbitrary simulation times.

To provide mission diversity, trajectories are sampled probabilistically rather than uniformly. Earth–Moon transfer trajectories are intentionally oversampled with a probability of 0.35 because

of their larger state variation and broader operating envelope. The remaining mission classes are sampled with approximately equal probabilities of 0.13 each. This approach produces broad mission coverage while maintaining an intentionally nonuniform dataset distribution.

4.2 Disturbance Injection and Recovery Trajectories

The dataset is designed primarily to capture corrective guidance behavior rather than merely nominal trajectory tracking. For each generated reference trajectory, a disturbance location is selected randomly from an intermediate portion of the trajectory. The implementation in `Dataset_Generator.py` selects the disturbance index after an initial set of reference samples while reserving sufficient trajectory duration for subsequent recovery behavior.

A position and velocity perturbation are added to the nominal reference state:

$$\mathbf{x}_{\text{perturbed}} = \mathbf{x}_{\text{ref}} + \Delta \mathbf{x}.$$

For each disturbance event, a position-error scaling magnitude is selected between

$$1.5 \text{ km} \leq \epsilon_p \leq 3.0 \text{ km},$$

and a velocity-error scaling magnitude is selected between

$$0.1 \text{ m/s} \leq \epsilon_v \leq 0.3 \text{ m/s}.$$

Individual Cartesian components are then sampled independently using these values, with reduced perturbation limits applied in the out-of-plane direction. This approach produces a controlled range of off-nominal states while preserving repeatability during dataset generation.

The perturbed spacecraft state is then propagated under the nonlinear CRTBP dynamics with the adaptive gain-scheduled PID-like controller and continuous disturbance model enabled. The perturbed spacecraft states generated during this recovery process are included directly as training samples. Therefore, the dataset contains both near-nominal tracking examples and off-nominal recovery states paired with the PID-like controller acceleration commands required to reduce trajectory errors. The resulting trajectory contains the state evolution and corrective thrust behavior produced by the PID-like policy. This process intentionally increases the presence of off-nominal states in the training distribution. Consequently, the model observes examples containing nominal tracking errors, recovery maneuvers, and varying corrective acceleration commands rather than learning only steady-state stationkeeping behavior.

4.3 Mission Metadata and Context Representation

Each generated spacecraft state is annotated with mission-level information before conversion into a training sample. The metadata generation process is implemented through `verify_and_get_dynamic_metadata()` in `Dataset_Generator.py`. When the spacecraft location satisfies predefined geometric criteria associated with Earth-orbit, Moon-orbit, halo, or DRO regions, the mission category is inferred dynamically from the current state. If the spacecraft location does not produce an unambiguous classification, the original trajectory label is retained as a fallback.

Each dataset entry records:

- **Mission Identification and Destination:** Unique trajectory identifier (`trajectory_id`), high-level mission objective, and final target destination.

- **Operating Regimes and Progress:** Dynamically assigned mission type, real-time mission progress percentage, and reference target state.
- **Perturbation Tracking:** Location and timestep index of the injected disturbance used for recovery generation.

The stored `trajectory_id` allows individual recovery trajectories to be reconstructed from the JSONL dataset. Additional debugging fields containing spacecraft and reference positions in kilometers provide a direct means of validating trajectory continuity after dataset generation.

4.4 Natural-Language State Representation

The spacecraft state is transformed into a structured natural-language prompt using the `generate_prompt_text()` function in `Dataset_Generator.py`. The prompt contains both physical quantities and mission context:

- **Kinematic State:** Current position and velocity in canonical units and engineering units, vector magnitudes, position azimuth/elevation, and velocity azimuth/elevation.
- **Reference Trajectory State:** Desired position and velocity targets together with real-time position and velocity tracking errors.
- **Mission Context:** Mission progress percentage, final destination information, and high-level mission objective.

Each prompt begins with the mission-level instruction:

User: You are an autonomous spacecraft controller operating in Earth-Moon synodic space, currently 60.2% complete with an Earth Orbit maneuver (Earth-centered parking orbit stationkeeping and maintenance.). Your current continuous physical state is as follows: Position: X-component is 0.037253 canonical units (14,320 km from the barycenter), Y-component is -0.054162 canonical units (-20,820 km), and Z-component is -0.001818 canonical units (-699 km). This results in a total position magnitude of 25,279 km, directed at an azimuth of 304.52 degrees and an elevation of -1.58 degrees. Velocity: X-component is 3.419306 canonical units (3498.5 m/s), Y-component is 2.751365 canonical units (2815.1 m/s), and Z-component is -0.013766 canonical units (-14.1 m/s). This yields a total synodic speed of 4490.5 m/s, pointing toward an azimuth of 38.8 degrees and an elevation of -0.18 degrees. Your target waypoint requires an X-position of 0.037253 canonical units (14,320 km), a Y-position of -0.054162 canonical units (-20,820 km), a Z-position of -0.001818 canonical units (-699 km), an X-velocity of 3.419306 canonical units (3498.5 m/s), a Y-velocity of 2.751365 canonical units (2815.1 m/s), and a Z-velocity of -0.013766 canonical units (-14.1 m/s). This yields a position tracking error magnitude of 0 km (directed toward an azimuth of 58.9 degrees and an elevation of -0.7 degrees) and a velocity tracking error magnitude of 0.00 m/s (directed toward an azimuth of 47.8 degrees and an elevation of -5.4 degrees). Your final mission destination magnitude is 65,560 km (0.170552 canonical units). Determine the precise thrust acceleration vector needed to correct this tracking offset and maintain nominal trajectory.

and concludes by requesting the continuous acceleration vector required to correct the current tracking condition. This representation was selected to provide compatibility with instruction-tuned LLMs while maintaining the numerical information necessary for spacecraft control. Each sample

is therefore presented as a conversational interaction between a user describing the spacecraft state and a PID-like assistant providing the corresponding control action.

4.5 Natural-Language Control Representation

The PID-like controller generates a continuous three-axis acceleration command:

$$\mathbf{u} = [u_x \quad u_y \quad u_z]^T.$$

The command is converted into a standardized assistant response containing:

- **Cartesian Vector Components:** Three-axis continuous acceleration values (u_x, u_y, u_z) .
- **Magnitude and Orientation:** Total acceleration magnitude $(\|\mathbf{u}\|)$ together with control azimuth and elevation.

Both canonical acceleration units and physical units (m/s^2) are included. The standardized response format is generated through `generate_response_text()` and enables a deterministic interface between foundation-model output and numerical spacecraft simulation. A representative response is:

Assistant: Command a continuous thrust acceleration vector of $+8.9141 \times 10^{-7} \text{ m/s}^2$ (0.000327 canonical units) along the X-axis, $+1.2422 \times 10^{-6} \text{ m/s}^2$ (0.000456 canonical units) along the Y-axis, and $-7.2122 \times 10^{-8} \text{ m/s}^2$ (-0.000026 canonical units) along the Z-axis. This applies a total physical thrust acceleration magnitude of $1.5306 \times 10^{-6} \text{ m/s}^2$, oriented toward an azimuth of 54.3 degrees and an elevation of -2.7 degrees.

This natural-language representation preserves the numerical precision of the adaptive gain-scheduled PID-like controller output while remaining compatible with supervised fine-tuning of conversational LLMs.

4.6 Dataset Composition

The final dataset contains 1,311,089 supervised prompt-response pairs generated from 600 complete spacecraft trajectories. Samples are extracted using a stride of two integration outputs to reduce redundancy while preserving the temporal structure of each recovery maneuver. The mission distribution is summarized in Table 2.

Mission Type	Samples	Percentage
Earth Orbit	604,780	46.1%
Earth–Moon Transfer	351,955	26.8%
Moon Orbit	173,294	13.2%
L_1 Halo Orbit	125,055	9.5%
L_2 Halo Orbit	42,173	3.2%
DRO	13,832	1.2%

Table 2: Mission-class distribution for the language-model training dataset.

The dataset provides coverage across multiple cislunar dynamical regimes, but the distribution is intentionally nonuniform. Earth-orbit and transfer trajectories represent the majority of samples,

while L_2 halo and DRO trajectories are comparatively less represented. This imbalance is a potential limitation because the learned policy may have less exposure to rare mission regimes and their associated control behaviors.

4.7 Dataset Verification

To evaluate dataset integrity, the stored JSONL samples were reconstructed into complete trajectory histories using the `Verify_Data.py` validation script. Samples were grouped by `trajectory_id` and sorted by trajectory progress before comparing the reconstructed spacecraft path with the corresponding reference trajectory.

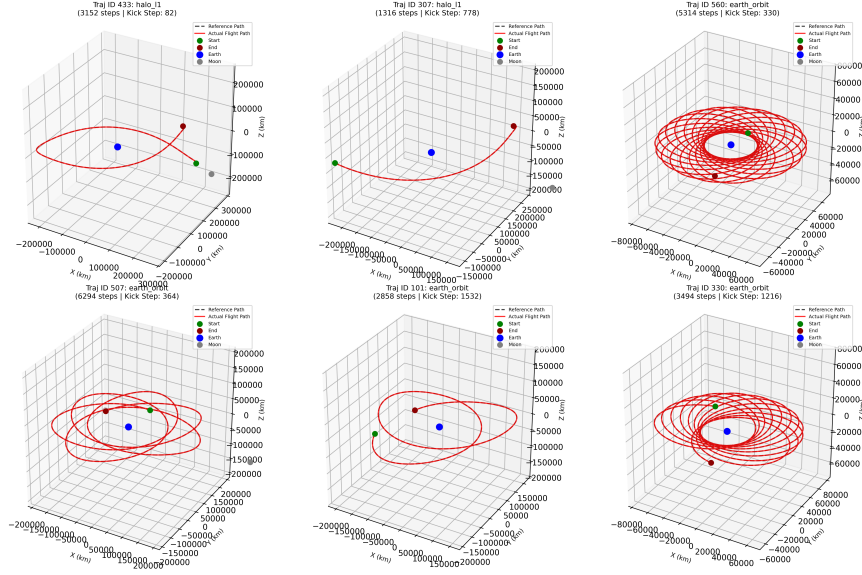


Figure 4: Reconstructed trajectories from the language-model dataset showing reference paths and controlled recovery trajectories across multiple mission classes.

The verification process confirmed that the dataset preserved trajectory continuity, mission labels, and disturbance-recovery histories. The reconstruction comparison produced a mean position discrepancy of approximately 0.03 km, a median discrepancy of 0.01 km, and a maximum observed discrepancy of 3.87 km. These values represent agreement between stored dataset states and the reconstructed PID-like controlled trajectories. They validate the quality of the controller but do not represent the performance of the later LLM controller. The resulting dataset provides a large-scale collection of high-resolution spacecraft guidance trajectories suitable for supervised fine-tuning of a LLM. The dataset quality reflects the fidelity of the PID-like controller trajectories from which it was generated, while later closed-loop simulations are required to evaluate how effectively the learned model reproduces this behavior.

5 Large Language Model Fine-Tuning

The foundation-model fine-tuning process adapts a pretrained language model to reproduce the PID-like spacecraft-guidance behavior generated in Section 3. The fine-tuned model learns to transform mission-aware natural-language descriptions into structured continuous thrust commands without

explicit access to the analytical control law. The resulting LoRA-adapted model provides the learned guidance policy evaluated during the closed-loop simulations presented in Section 6.

Rather than training a dedicated numerical neural-network controller, this work investigates whether an instruction-tuned LLM can learn a spacecraft-control representation through supervised imitation learning. The objective of this stage is therefore to evaluate foundation-model adaptation and command-generation capabilities, while closed-loop trajectory performance is assessed separately.

5.1 Base Model Selection

The LLM selected for this work is Meta’s Llama 3.1 8B Instruct model [4]. This model was selected because it provides a balance between model capacity, instruction-following capability, and computational requirements while remaining practical for parameter-efficient adaptation on a large spacecraft-guidance dataset.

Unlike conventional machine-learning control policies that operate directly on numerical state vectors, Llama 3.1 is designed to process conversational inputs and generate text-based outputs. This architecture is compatible with the language-based spacecraft-control representation developed in Section 4, where each example is structured as a user description of spacecraft state, mission objective, reference trajectory, and tracking condition, followed by a generated thrust command response.

5.2 Parameter-Efficient LoRA Adaptation

A full fine-tuning procedure would require updating all parameters of the 8-billion-parameter model. Instead, this work uses Low-Rank Adaptation (LoRA) to introduce a smaller set of trainable parameters while keeping the original pretrained weights fixed. The adapted model weights can be conceptually represented as

$$W = W_0 + \Delta W,$$

where W_0 represents the original pretrained weights and ΔW represents the learned low-rank correction.

The LoRA configuration used in this study employs a rank of $r = 16$, with scaling parameter $\alpha = 32$, and dropout rate $p = 0.05$. LoRA adapters are applied to both transformer attention and feedforward projection layers:

$$q_{\text{proj}}, k_{\text{proj}}, v_{\text{proj}}, o_{\text{proj}},$$

and

$$\text{gate}_{\text{proj}}, \text{up}_{\text{proj}}, \text{down}_{\text{proj}}.$$

This approach allows the model to acquire spacecraft-guidance behavior while substantially reducing memory requirements and training cost relative to full parameter optimization.

5.3 Supervised Instruction Fine-Tuning

The model is trained using supervised instruction fine-tuning with the PID-like controller dataset generated in Section 4. The adaptive gain-scheduled PID-like controller serves as the teacher policy, while the LLM learns to approximate the mapping between mission-aware spacecraft descriptions and corresponding acceleration commands. Each training example is converted into the native Llama conversational format using the tokenizer chat template. The resulting token sequence is optimized using a causal language-modeling objective, where the model learns to predict the PID-like

response tokens conditioned on the spacecraft-state prompt. The learned policy can be represented as

$$\pi_{\text{FM}} : (s, r, m, p) \rightarrow u,$$

where:

- s denotes the current spacecraft state,
- r denotes the desired reference trajectory state,
- m denotes mission context,
- p denotes mission progress,
- u denotes the desired continuous thrust acceleration command.

Importantly, the model is not directly optimized using spacecraft trajectory error, stability margin, or propellant consumption. Nor does it include the internal integral state or adaptive gain state of the adaptive gain-scheduled PID-like controller. Instead, these behaviors are learned indirectly through imitation of the PID-like controller responses contained within the supervised dataset. Therefore, successful language-model training does not directly imply successful spacecraft control performance.

5.4 Training Configuration

The complete dataset described in Section 4 was loaded using the Hugging Face `datasets` framework and divided into training and evaluation subsets using a fixed 95%/5% split with random seed 42. The held-out evaluation subset was limited to the first 5,000 samples from the evaluation partition to reduce evaluation overhead while maintaining a consistent comparison throughout training.

All prompt-response pairs were formatted using the native Llama conversational template before being passed to the supervised fine-tuning trainer. Training was performed using the Hugging Face TRL `SFTTrainer` framework [5] with bfloat16 precision, sequence packing, gradient accumulation, and gradient checkpointing enabled. The major training parameters are summarized in Table 3.

Category	Parameter	Value
Architecture	Base Model	Llama 3.1 8B Instruct
	LoRA Rank (r) / Alpha (α)	16 / 32
	LoRA Dropout (p)	0.05
	Target Modules	Attention & MLP Projections
Dataset	Total Dataset Size	1,311,089 samples
	Train / Eval Split	95% / 5% (5,000 eval samples)
	Maximum Sequence Length	512 tokens
Optimization	Learning Rate	2×10^{-4} (Cosine decay)
	Warmup Ratio	0.03
	Effective Batch Size	32
	Completed Epochs	2.57
	Precision	bfloat16

Table 3: Foundation-model fine-tuning hyperparameter configuration.

5.5 Training Convergence and Performance

Training and evaluation metrics demonstrate stable convergence throughout the fine-tuning process. Both training and evaluation losses decrease rapidly during early optimization before gradually approaching similar asymptotic values, as illustrated in Figure 5. The close agreement between the two curves indicates that the adapted model learned a consistent representation across the training and held-out dataset partitions without evidence of severe overfitting.

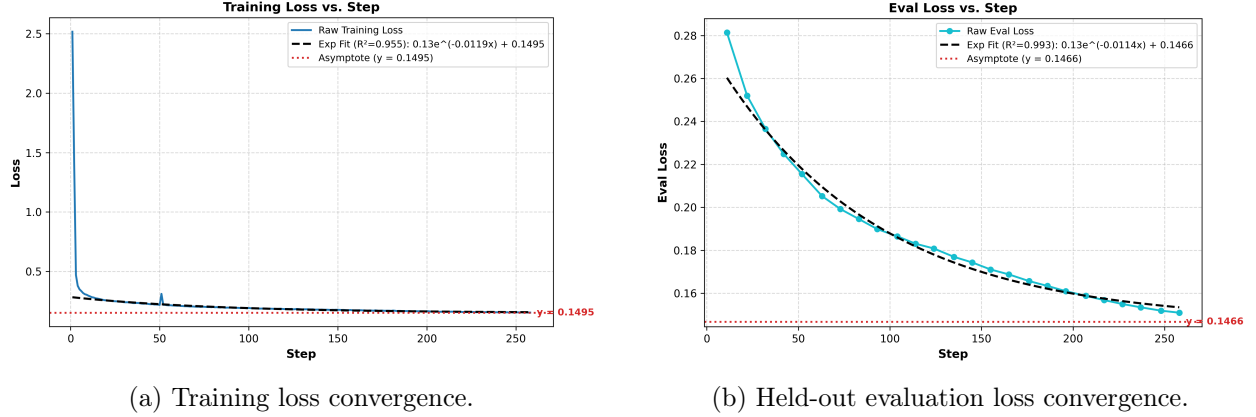


Figure 5: Training and evaluation loss convergence curves during supervised fine-tuning.

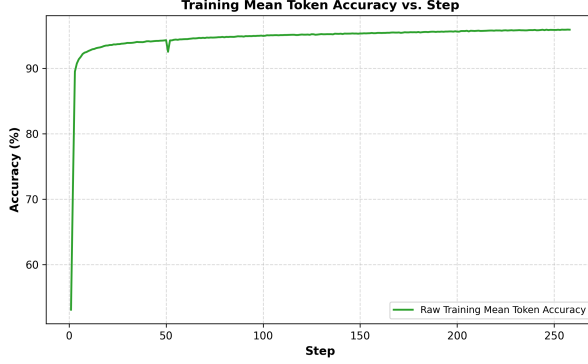
The initial training loss was approximately 2.5175, while the final recorded training loss reached 0.1526. The initial evaluation loss was 0.2813, and the final evaluation loss stabilized at 0.1509. The close agreement between final training and evaluation losses indicates consistent learning across the two dataset partitions. These results demonstrate successful adaptation of the model to the structured language-control representation. However, they do not by themselves guarantee stable spacecraft performance when the model is deployed as a controller within nonlinear CRTBP dynamics.

The loss convergence behavior was further characterized using exponential-decay fitting:

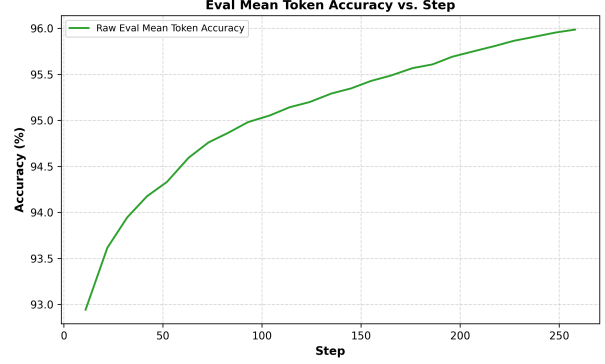
- **Training Loss Fit:** $R^2 = 0.9552$ with an estimated asymptotic limit of 0.1495.
- **Evaluation Loss Fit:** $R^2 = 0.9926$ with an estimated asymptotic limit of 0.1466.

These results indicate that the optimization process followed a consistent convergence trend during fine-tuning.

Token-level accuracy was also monitored, as seen in Figure 6, to evaluate how effectively the model reproduced the structured control response format.



(a) Training token prediction accuracy.



(b) Evaluation token prediction accuracy.

Figure 6: Training and evaluation token-level accuracy during supervised fine-tuning.

The final training mean token accuracy reached 95.91%, while the final held-out evaluation mean token accuracy reached 95.99%, improving from an initial evaluation accuracy of approximately 92.94%. The similarity between training and evaluation accuracy indicates that the model learned a consistent representation of the structured PID-like control response format across the evaluated dataset partitions.

It is important to note that token-level accuracy measures the model’s ability to reproduce the textual structure and numerical formatting of the training responses, rather than the physical accuracy or effectiveness of the generated spacecraft commands. Small token-level deviations can correspond to significant changes in acceleration magnitude or direction, and therefore a high token accuracy does not necessarily imply accurate control action reproduction, trajectory stability, disturbance rejection capability, or successful closed-loop spacecraft guidance. Consequently, these metrics demonstrate the fidelity of language-model imitation at the representation level but should not be interpreted as direct measures of spacecraft-control performance. The fine-tuning performance and convergence metrics are summarized in Table 4.

Evaluation Metric	Initial	Final	Δ / Improvement
Training Loss	2.5175	0.1526	−2.3649
Evaluation Loss	0.2813	0.1509	−0.1304
Training Token Accuracy	—	95.91%	—
Evaluation Token Accuracy	92.94%	95.99%	+3.05%

Table 4: Supervised fine-tuning convergence and token accuracy performance.

The fine-tuning results demonstrate that the LoRA-adapted LLM successfully learned the structured mapping between mission-aware spacecraft descriptions and generated thrust commands. The resulting model is subsequently evaluated as a closed-loop guidance policy in Section 6, where trajectory stability, disturbance response, and physical control performance are assessed directly.

6 Closed-Loop Large Language Models Guidance Architecture and Evaluation

This section evaluates the fine-tuned LLM as an active spacecraft controller operating directly within the nonlinear simulation loop. The closed-loop architecture combines mission-aware prompt gener-

ation, foundation-model inference, text-to-command conversion, safety conditioning, and numerical propagation of the disturbed Earth–Moon CRTBP dynamics. The objective of this evaluation is not only to determine whether the language-conditioned policy can generate physically feasible acceleration commands, but also to determine whether imitation learning preserves the trajectory-tracking and disturbance-rejection characteristics of the PID-like controller. The results therefore evaluate both the capability and limitations of direct LLM-based spacecraft guidance.

6.1 Mission-Aware Prompt Construction

At each guidance update, the current spacecraft state is converted into the same structured natural-language representation used during dataset generation. Maintaining a consistent state-description format between training and deployment reduces distribution differences between supervised learning and closed-loop inference. The deployed controller receives:

- Current spacecraft position and velocity,
- Desired reference trajectory state,
- Position and velocity tracking errors,
- Mission progress,
- Dynamically evaluated mission type and objective,
- Final mission destination.

Before prompt generation, the spacecraft state is evaluated using the dynamic mission-classification procedure introduced in Section 4. This allows the controller to update mission context according to the spacecraft’s current operating region rather than relying exclusively on the initial trajectory label. The resulting prompt representation is expressed as

$$\mathcal{P} = f(\mathbf{x}, \mathbf{x}_{\text{ref}}, \mathbf{x}_{\text{err}}, m, p),$$

where $\mathbf{x}$ is the current spacecraft state, $\mathbf{x}_{\text{ref}}$ is the reference state, $\mathbf{x}_{\text{err}}$ is the tracking error, m is mission context, and p is mission progress. The prompt is then formatted using the native Llama conversational template before model inference.

6.2 Language Model Inference and Control Generation

The fine-tuned Llama 3.1 LLM generates a text response containing the desired three-axis acceleration command:

$$\mathbf{u}_{\text{LLM}} = \pi_{\text{LLM}}(\mathcal{P}).$$

Inference is performed deterministically using greedy generation (`do_sample = False`) to provide repeatable control behavior between simulations.

The generated response contains the acceleration components along the synodic coordinate axes in canonical and physical units. To reduce computational cost, the LLM is not queried at every integration step. Under nominal conditions, inference is performed every ten simulation steps. When position error exceeds 5 km, the update interval is reduced to every two steps to increase corrective responsiveness.

6.3 Command Extraction and Physical Validation

Because the LLM generates text rather than a numerical control vector, an additional parsing stage converts the response into a valid spacecraft command. The `parse_llm_canonical_accel()` function first searches for explicitly labeled X-, Y-, and Z-axis acceleration values. If structured parsing fails, a fallback numerical extraction method attempts to recover a three-component vector. The resulting command is represented as

$$\mathbf{u} = \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}.$$

The command magnitude is constrained according to

$$\|\mathbf{u}\| \leq u_{\max},$$

where $u_{\max} = 0.01 \text{ m/s}^2$. If parsing fails completely, the controller maintains the previous valid command rather than applying an undefined input. These mechanisms provide a deterministic interface between language generation and numerical spacecraft simulation.

6.4 Safety Conditioning and Command Smoothing

Before application to the spacecraft dynamics, the parsed command is passed through a smoothing and rate-limiting layer. Exponential smoothing is applied as

$$\mathbf{u}_{\text{filt}} = \alpha \mathbf{u}_{\text{LLM}} + (1 - \alpha) \mathbf{u}_{\text{prev}},$$

with $\alpha = 0.25$. The command transition is calculated as

$$\Delta \mathbf{u} = \mathbf{u}_{\text{filt}} - \mathbf{u}_{\text{prev}}.$$

If the transition magnitude exceeds the prescribed value 2.0×10^{-5} , the command is reduced before application.

These layers ensure that generated commands remain physically compatible with spacecraft constraints. However, they only constrain the output of the model and do not actively stabilize an incorrect guidance prediction.

6.5 Closed-Loop Guidance Cycle

The closed-loop guidance process is executed as follows:

1. Evaluate the current spacecraft and reference states.
2. Generate the mission-aware natural-language prompt.
3. Perform deterministic LLM inference.
4. Parse the textual response into a numerical acceleration vector.
5. Apply acceleration limits, smoothing, and rate constraints.
6. Propagate the spacecraft using disturbed CRTBP dynamics.
7. Record tracking performance and continue the guidance cycle.

Unlike the PID-like controller, the LLM does not directly use local stability information, adaptive gains, state-transition matrices, or integral compensation.

6.6 Large Language Model Evaluation

The LLM controller was evaluated across five cislunar mission classes:

- Earth Orbit,
- Moon Orbit,
- Earth–Moon Transfer,
- L_1 / L_2 Halo Orbit,
- Distant Retrograde Orbit.

Each simulation began from a nominal reference trajectory. Continuous disturbances were applied using the same deterministic formulation used during dataset generation. An additional random position and velocity perturbation was introduced at simulation step 1000 to evaluate recovery behavior.

The following quantities were recorded:

- Spacecraft state history,
- Reference trajectory history,
- LLM acceleration commands,
- Position and velocity tracking errors,
- Disturbance events,
- Total cumulative ΔV .

6.7 Evaluation Metrics

Performance was evaluated using position error, velocity error, cumulative ΔV , command-limit compliance, and numerical completion status. The total control expenditure was calculated as

$$\Delta V = \int_{t_0}^{t_f} \|\mathbf{u}(t)\| dt.$$

These metrics distinguish between language-model command generation quality and spacecraft-level trajectory performance.

6.8 Comparison of Controller Information Structures

The adaptive controller described in Section 3 provides the PID-like reference policy used for dataset generation. The purpose of the comparison is to evaluate how effectively a language-conditioned model can approximate PID-like guidance behavior after supervised training.

The PID-like controller has explicit access to local CRTBP dynamics, adaptive gain scheduling, integral compensation, actuator constraints, and continuously updated tracking-error information. In contrast, the LLM guidance policy receives only the observable spacecraft state, reference trajectory state, tracking error, and mission context, without access to the expert controller’s internal integral state, adaptive gain state, or previous control history. Therefore, the PID-like controller operates with a more complete feedback information structure than direct LLM inference. In addition to differences in policy representation and available internal information, the comparison also

reflects differences in control-update frequency. The PID-like expert controller operates continuously during numerical propagation, while the LLM guidance policy uses discrete sampled inference updates with additional output-conditioning layers. Therefore, the comparison evaluates not only the ability of imitation learning to reproduce expert behavior, but also the effects of applying that behavior through a lower-frequency language-model-based control architecture.

6.9 Large Language Model Controller Performance

The LLM controller generated valid bounded acceleration commands across all evaluated mission classes. The resulting trajectories remained numerically bounded and the simulations completed without numerical failure. However, the tracking performance was substantially weaker than the PID-like controller baseline.

6.9.1 Earth Orbit Missions

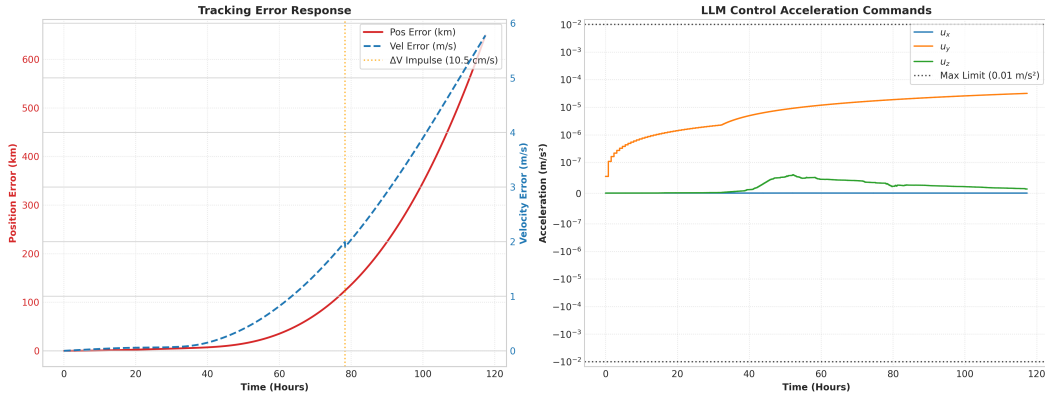


Figure 7: LLM-controlled Earth Orbit trajectory tracking and disturbance response.

Earth-orbit operation provided the most favorable dynamical environment for the LLM controller. Tracking error remained small during the early portion of the simulation, increasing gradually as the mission progressed. Following the injected disturbance, the final position error grew to approximately 500 km, with a final velocity error near 4.9 m/s. The LLM generated smooth commands, but the corrective action was insufficient to restore the precision achieved by the PID-like controller.

6.9.2 Moon Orbit Missions

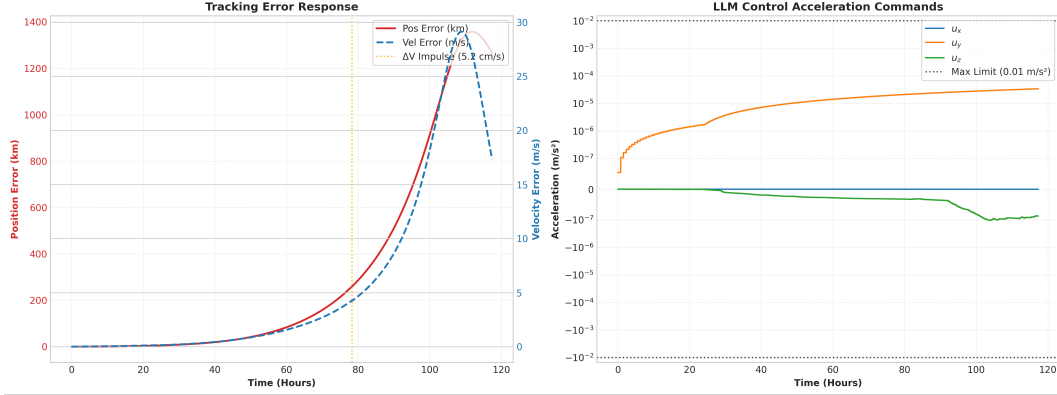


Figure 8: LLM-controlled lunar orbit trajectory tracking and disturbance response.

The lunar-orbit case presented the largest tracking degradation among the evaluated missions. After the disturbance was applied, the position error increased continuously and reached approximately 1338 km, while velocity error approached 29 m/s. The command magnitude remained bounded, indicating that the performance limitation was not caused by excessive thrust generation or numerical instability. Instead, the predicted corrections were not sufficiently effective at counteracting the accumulated trajectory deviation.

6.9.3 Earth–Moon Transfer Missions

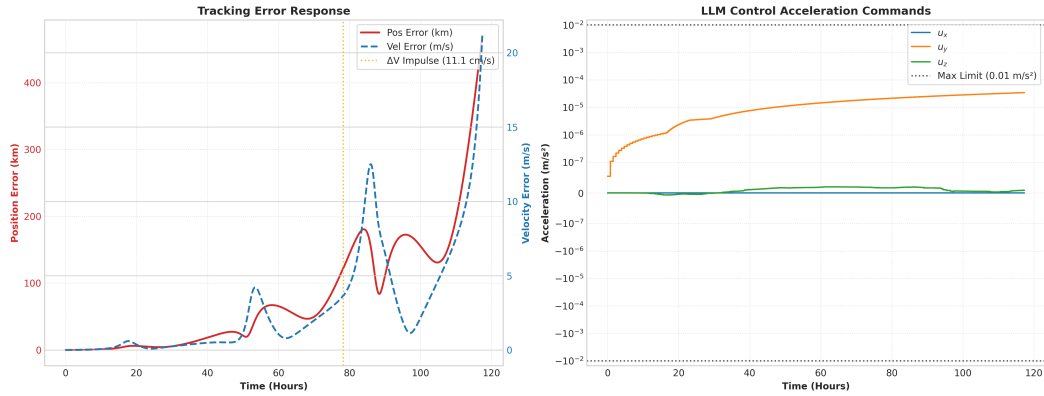


Figure 9: LLM-controlled Earth–Moon transfer trajectory tracking and disturbance response.

The transfer case showed the strongest relative tracking performance among the five LLM-controlled missions. The trajectory remained close to the desired transfer path for much of the evaluation period, with the final position error reaching approximately 189 km. Although this result is substantially better than the orbit and halo cases, it remains significantly less accurate than the PID-like controller performance.

6.9.4 Halo Orbit Missions

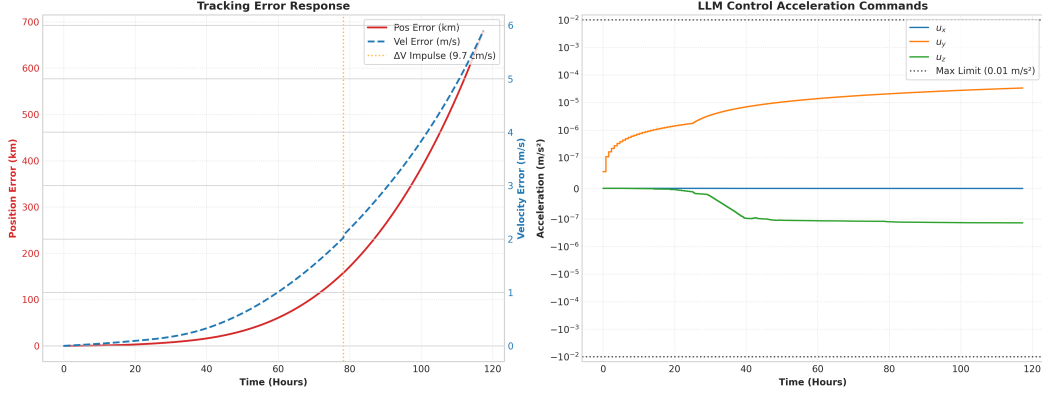


Figure 10: LLM-controlled L1 halo orbit tracking and disturbance response.

The L_1 halo trajectory introduced additional sensitivity due to the underlying libration-point dynamics. The LLM controller produced bounded commands and remained in the general halo-orbit region, but the position error increased to approximately 533 km by the end of the simulation. The result demonstrates that recognizing the halo-orbit mission context was not sufficient to reproduce the precise timing and magnitude of corrections required for stationkeeping near an unstable equilibrium region.

6.9.5 Distant Retrograde Orbit Missions

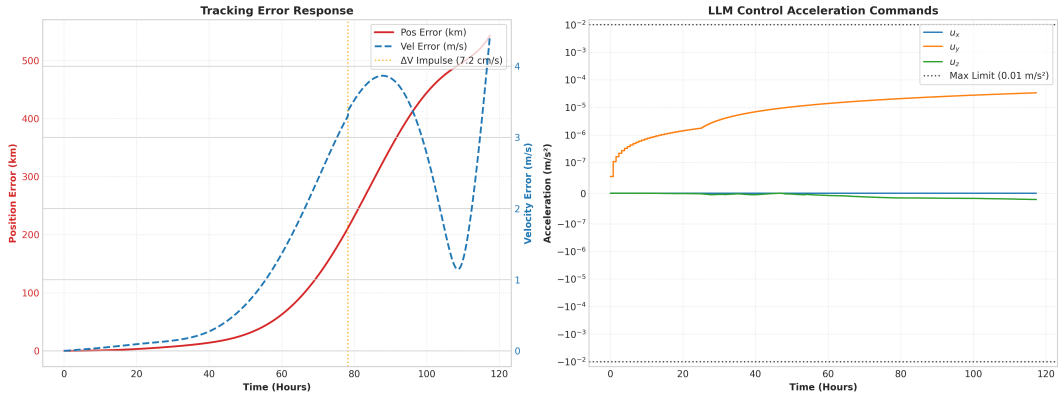


Figure 11: LLM-controlled distant retrograde orbit tracking and disturbance response.

The DRO scenario exhibited similar behavior. The trajectory remained bounded within the intended lunar dynamical region, but tracking error accumulated over time and reached approximately 496 km. Although DRO trajectories possess favorable natural stability properties, maintaining accurate reference tracking still requires appropriately timed corrective actions, which the current LLM policy did not consistently provide.

6.10 Aggregate Control Performance

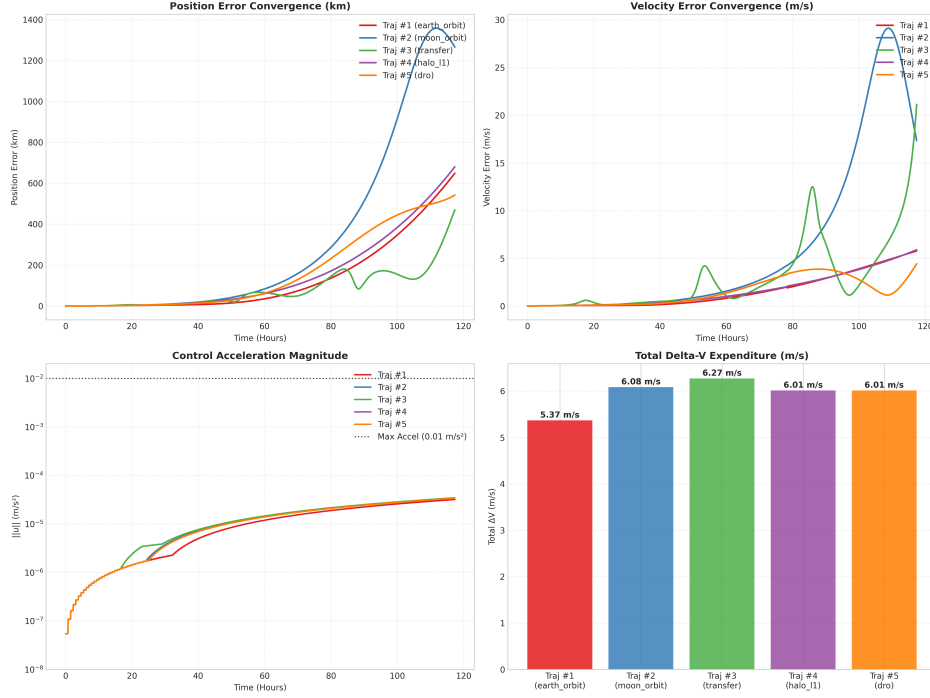


Figure 12: Aggregated LLM performance across multiple cislunar trajectory classes, including tracking error, velocity error, acceleration magnitude, and total ΔV .

The similar ΔV values, shown in the third column of Table 5, indicate that the controller produced consistent and bounded control effort across all cases. The cumulative control expenditures were:

Mission	Final Position Error (km)	Final Velocity Error (m/s)	Total ΔV (m/s)
Earth Orbit	500	4.94	5.37
Moon Orbit	1338	28.98	6.08
Transfer	189	7.28	6.27
L_1 Halo	533	4.87	6.01
DRO	496	1.22	6.01

Table 5: Closed-loop LLM guidance performance across evaluated cislunar mission classes.

However, the accumulated tracking errors show that the LLM did not allocate correction authority in a manner consistent with the increasing difficulty of the mission environments.

6.11 Summary of Closed-Loop Findings

The closed-loop simulations demonstrate that a LLM can generate syntactically valid and physically bounded spacecraft commands across multiple cislunar trajectory classes. The safety-conditioning layers successfully prevented command formatting failures and excessive acceleration commands. However, the simulations also reveal a significant gap between language imitation accuracy and spacecraft-control effectiveness. Increasing position and velocity errors occurred in all evaluated missions, particularly following disturbance injection, despite maintaining bounded acceleration

commands. The results indicate that the learned policy captured aspects of expert behavior but did not reproduce the internal feedback mechanisms of the analytical controller, including integral-state accumulation and adaptive gain scheduling.

7 Discussion

The closed-loop evaluations demonstrate that LLMs can learn meaningful spacecraft-control representations while also revealing the limitations of using language-model inference as an LLM-driven closed-loop guidance policy.

7.1 Strengths of Language-Conditioned Guidance

The primary strength of the proposed framework is the ability to combine spacecraft state, mission objectives, trajectory information, and tracking conditions within a single representation. Unlike conventional controllers that primarily operate on numerical state vectors, the language-conditioned approach provides a unified interface capable of incorporating semantic mission context alongside physical spacecraft information.

The fine-tuned model generated structured acceleration commands across multiple cislunar mission classes using a common architecture rather than requiring separate learned policies for each trajectory family. This demonstrates the potential of LLMs as flexible decision-making systems capable of representing guidance behavior across diverse spacecraft operating conditions.

The semantic interface also provides a pathway for incorporating additional contextual information, including spacecraft health status, navigation uncertainty, mission priorities, operational constraints, and other mission-level information that may be difficult to integrate into fixed-structure control architectures.

The results further demonstrate that the model learned meaningful relationships between spacecraft conditions and corrective guidance actions. Although the learned policy did not reproduce the tracking performance of the PID-like controller, the generated commands remained structured, bounded, and responsive to changes in spacecraft state and mission context.

7.2 Limitations of Large Language Models for Spacecraft Control

The current results identify several limitations:

- **Loss of explicit feedback structure:** The LLM learns an approximation of the PID-like controller’s input-output mapping but does not explicitly retain the underlying feedback mechanisms, including adaptive gain changes, integral-state behavior, or the continuous relationship between tracking error and corrective acceleration.
- **Accuracy-performance mismatch:** Although the model achieved approximately 96% token accuracy during fine-tuning, this did not translate into high-level trajectory tracking. Language-model prediction accuracy alone is therefore insufficient as a measure of spacecraft-control effectiveness.
- **Limited temporal awareness:** The current formulation provides the model with instantaneous spacecraft and reference conditions but does not explicitly include previous state history or prior commands. This may limit the ability to identify error trends and determine the appropriate timing and magnitude of corrective actions.

- **Dependence on training distribution:** The learned policy is dependent on the trajectory families and disturbance scenarios represented in the dataset. Rare mission regimes, larger disturbances, or unseen operating conditions may therefore result in degraded performance.
- **Computational cost:** Foundation-model inference requires substantially more onboard computational capability than traditional feedback laws. Practical deployment would require optimization through model compression, efficient inference methods, or specialized spacecraft computing hardware.
- **Imitation limitations:** Imitation learning transfers the strengths and limitations of the PID-like control trajectories into the learned policy. Missing recovery scenarios, insufficiently represented trajectory regions, or suboptimal PID-like behavior may therefore limit performance.

7.3 Comparison with Classical Guidance Approaches

Classical spacecraft guidance methods remain the preferred approach for precision operations because they provide predictable behavior, efficient execution, and direct relationships between system error and corrective response. The PID-like controller developed in this work demonstrates these advantages by explicitly incorporating CRTBP dynamics, adaptive gain scheduling, integral compensation, and actuator constraints.

The language-conditioned approach should therefore be viewed as a different control representation rather than a direct replacement for traditional guidance methods. LLMs provide flexibility by combining mission context and numerical state information within a unified framework, while classical controllers provide explicitly designed feedback behavior.

The results show that this flexibility currently comes with reduced trajectory-control accuracy. The LLM generated feasible commands, but the resulting closed-loop behavior did not match the correction timing, disturbance-rejection performance, and long-duration tracking capability demonstrated by the PID-like controller. This suggests that the main challenge is not command generation alone, but learning the temporal and dynamic characteristics required for reliable spacecraft guidance.

7.4 Implications for Autonomous Cislunar Operations

The results indicate that LLMs may become useful components of future autonomous spacecraft systems, particularly for interpreting mission context, adapting to changing objectives, and integrating heterogeneous operational information. The present study establishes an important boundary for fully LLM-based spacecraft control: a language model can learn meaningful guidance behavior and generate physically constrained commands, but the current approach does not yet achieve the consistency required for precision long-duration trajectory tracking.

Achieving fully autonomous LLM-based spacecraft guidance will likely require advances in temporal reasoning, control-oriented training objectives, uncertainty handling, and evaluation methods focused directly on spacecraft-level performance rather than language-model metrics alone.

7.5 Future Work

Several research directions emerge from the limitations identified during the closed-loop evaluation:

1. **Augmented Controller-State Representation:** Future work should investigate language representations that explicitly include latent variables used by adaptive controllers. Candidate inputs include accumulated integral error, adaptive gain magnitude, local instability metrics,

saturation state, previous commands, and uncertainty estimates. Including these variables would allow the LLM to observe the full decision state of the expert policy rather than infer it from instantaneous spacecraft conditions.

2. **Dataset Balancing:** The training dataset should be expanded and balanced around the most difficult mission regimes. Increasing representation of dynamically sensitive cases (L_2 halo, halo recovery, and DRO) and including larger varieties of disturbance scenarios may improve performance in regimes showing the greatest tracking degradation.
3. **Beyond Imitation Learning:** Future studies should explore spacecraft-level optimization objectives beyond supervised imitation. Reinforcement learning, constrained policy optimization, or hybrid loss functions incorporating position error, control effort, and safety constraints could allow the learned policy to move beyond direct reproduction of PID-like responses.
4. **High-Fidelity Validation:** Additional validation is required before considering practical spacecraft deployment. Future work should include higher-fidelity gravitational models, navigation uncertainty, actuator response dynamics, hardware-in-the-loop testing, formal safety verification, and comparison against optimized classical guidance strategies.

8 Conclusions

This paper investigated the feasibility and limitations of using a LLM as a language-conditioned spacecraft guidance policy within the Earth–Moon Circular Restricted Three-Body Problem. A complete framework was developed integrating nonlinear CRTBP simulation, an adaptive gain-scheduled PID-like controller, large-scale natural-language trajectory generation, LoRA-based foundation-model fine-tuning, and direct closed-loop evaluation.

The adaptive gain-scheduled PID-like controller generated physically consistent guidance trajectories across multiple cislunar trajectory classes, including Earth Orbit, lunar orbit, Earth-Moon transfer, L_1 halo, and DRO trajectories. These trajectories were transformed into a dataset containing approximately 1.31×10^6 prompt-response pairs, enabling the LLM to learn a structured representation of spacecraft state, mission context, tracking condition, and continuous acceleration commands. Fine-tuning results demonstrated that the Llama 3.1 8B Instruct model successfully learned the language-based representation of the PID-like guidance policy, achieving approximately 96% held-out token-level accuracy with stable training and evaluation convergence.

However, the closed-loop evaluation demonstrated that accurate language-model imitation does not directly guarantee effective spacecraft-level guidance performance. The deployed LLM controller generated syntactically valid and physically bounded acceleration commands across all evaluated mission classes. However, trajectory errors increased following disturbance injection, indicating that the model captured portions of the PID-like guidance behavior without fully reproducing the correction timing, temporal consistency, and disturbance-rejection behavior demonstrated by the original feedback controller. The primary conclusion of this work is that LLMs can learn meaningful language-conditioned spacecraft guidance representations, but reproducing an adaptive nonlinear controller requires a language representation that captures the complete information state governing the guidance policy. In particular, the results identify an important gap between learning a control policy representation and reproducing the full closed-loop behavior required for reliable autonomous spacecraft operation.

Future progress toward fully autonomous LLM-based spacecraft guidance will require improved temporal memory, trajectory-aware training, control-oriented optimization objectives, uncertainty handling, and more extensive evaluation across increasingly realistic spacecraft environments.

9 Acknowledgments

This research was made possible through the ACCESS program and the Delta advanced computing and data resources provided through the National Center for Supercomputing Applications (NCSA). This work used the Delta system through ACCESS allocation CIS261093, which provided the computational resources necessary for large-scale dataset generation, foundation-model fine-tuning, and closed-loop spacecraft simulations. Thank you to Dr. Hiroyasu Tsukamoto for providing continued guidance, feedback, and recommendations throughout the development of this work. The technical discussions and insightful suggestions contributed substantially to the refinement of the methodology, analysis, and overall execution of this research.

References

- [1] J. E. Prussing and B. A. Conway, *Orbital Mechanics*. New York: Oxford University Press, 1993.
- [2] E. Lavretsky and K. A. Wise, *Robust and Adaptive Control: With Aerospace Applications*. London: Springer-Verlag, 2012.
- [3] E. M. Zucchelli, D. Wu, J. Briden, C. Hofmann, V. Rodriguez-Fernandez, and R. Linares, “Fine-Tuned Language Models as Space Systems Controllers,” *arXiv preprint arXiv:2501.16588*, 2025.
- [4] Meta AI, *Llama 3.1 8B Instruct*, 2024. Available: <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>. [Accessed: Aug. 25, 2026].
- [5] Hugging Face, *TRL: Transformer Reinforcement Learning Library and SFTTrainer Documentation*, 2024. Available: https://huggingface.co/docs/trl/sft_trainer. [Accessed: Aug. 25, 2026].
- [6] National Center for Supercomputing Applications, *Delta Advanced Computing and Data Resource*, 2022. Available: <https://delta.ncsa.illinois.edu>. Supported by NSF Award No. OAC-2005572 and the State of Illinois.
- [7] Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support, *ACCESS Advanced Computing and Data Resources*, 2024. Available: <https://access-ci.org>. Supported by NSF Grant Nos. 2138259, 2138286, 2138307, 2137603, and 2138296.